# RHealth: A R Toolkit for Deep Learning in Healthcare

**Ji Song**[1*], **Zhixia Ren**[1*], **Zhenbang Wu**[2], **John Wu**[2], **Chaoqi Yang**[2], **Yinghao Zhu**[1,3], **Wen Tang**[4], **Jimeng Sun**[2], **Ewen M Harrison**[5], **Liantao Ma**[1†], **Junyi Gao**[5,6†]
[1]National Engineering Research Center for Software Engineering, Peking University
[2]University of Illinois Urbana-Champaign
[3]School of Computing and Data Science, The University of Hong Kong
[4]Peking University Third Hospital
[5]Centre of Medical Informatics, The University of Edinburgh
[6]Health Data Research UK

***Abstract*—Machine learning for electronic health records (EHR) is advancing rapidly and already underpins risk stratification, readmission and mortality prediction, and decision support, yet reliable translation still stalls on fragmented data pipelines, inconsistent medical-code handling, and hard-to-reproduce evaluation—barriers that especially hinder R-centric clinical teams. Despite impressive methodological gains in temporal modeling, attention mechanisms, and strong classical baselines, most turnkey toolchains live in Python; as a result, many healthcare researchers and clinical data scientists working in R lack a single, integrated path from raw multi-table EHR to calibrated, auditable models. We address this gap with RHealth, an open-source, R-native toolkit that plays the role of an end-to-end conductor: from data harmonization and medical-code normalization to task specification, model training, and standardized reporting. Concretely, RHealth provides adapters for widely used public datasets (e.g., MIMIC-III/IV, eICU), utilities to traverse and map ICD-9/10 and CCS codes, task templates for common outcomes (mortality, 30-day readmission, length of stay), and a modeling stack that—at this development stage—supports standard recurrent baselines (e.g., RNN) and offers an extensible interface for user-defined architectures under active development, all evaluated with reproducible splits, AUROC/AUPRC, and calibration diagnostics. By packaging the full pipeline—from data to evaluation granularity—into modular, composable components, RHealth lowers the entry barrier for R users, reduces "glue code," and promotes transparent, people-centric experimentation that can also serve as a trustworthy upstream substrate for LLM-enabled applications. To our knowledge, it is among the first comprehensive, integrated deep-learning toolkits for EHR in the R ecosystem. The code and documentation will be released after the double-blind review process.**

***Index Terms*—EHR, R, toolkit, machine learning, deep learning**

## I. Introduction

Machine learning—particularly deep learning—has become central to predictive modeling with electronic health records (EHR), enabling risk stratification, early deterioration detection, and clinical decision support. Yet building reliable models in high-stakes settings still faces well-documented obstacles: irregular and sparse longitudinal signals, heterogeneous coding systems, label noise, and reproducibility gaps across institutions and studies [1], [2]. Methodological progress has been swift: gated sequence models such as LSTM and its time-aware variants capture long-range temporal dependencies and irregular sampling in clinical trajectories [3]–[5]; attention mechanisms improve both accuracy and interpretability in patient-level prediction (e.g., RETAIN) [6]; stage-aware and multimodal designs further integrate disease progression and continuous monitoring signals (e.g., StageNet, RAIM) [7], [8]. In parallel, clinical NLP advances—CAML, LAAT, and MultiResCNN—have raised the bar for automatic ICD coding and text-driven phenotyping, showcasing the value of representation learning for non-structured notes [9]–[11]. Even for physiological signals such as ECG, deep residual and bidirectional recurrent architectures have approached expert-level performance on arrhythmia detection, underscoring deep learning's potential breadth in healthcare analytics [12]. Classical baselines like Random Forests and XGBoost remain important points of comparison in tabular EHR regimes, highlighting the need for unified pipelines that support both traditional and neural approaches with equal rigor [13], [14].

Amid this progress, tooling has not kept pace for all user communities. Python ecosystems have matured (e.g., PyHealth) into end-to-end libraries that standardize data ingestion, task specification, modeling, and evaluation, helping alleviate fragmentation and promoting comparability across benchmarks [15]. However, many healthcare researchers and clinical data scientists work primarily in R, where no widely adopted, integrated deep learning toolkit for EHR has existed. This gap matters for a people-centric AI agenda: if domain experts cannot efficiently prototype and audit models in their native analytical environment, we slow down hypothesis testing, diminish transparency, and risk brittle, non-reproducible workflows. Moreover, the theme of this workshop—Advancing Data for Better Health: Reliable LLM Application in People-Centric Healthcare—emphasizes that trustworthy LLMs depend on upstream data curation and robust supervision signals. In practice, that means accessible toolchains for clean, well-coded, and well-evaluated structured data remain foundational, even (and especially) when downstream systems incorporate LLMs.

We introduce RHealth, an open-source R-native deep learn-

* Equal contribution, † Corresponding authors.

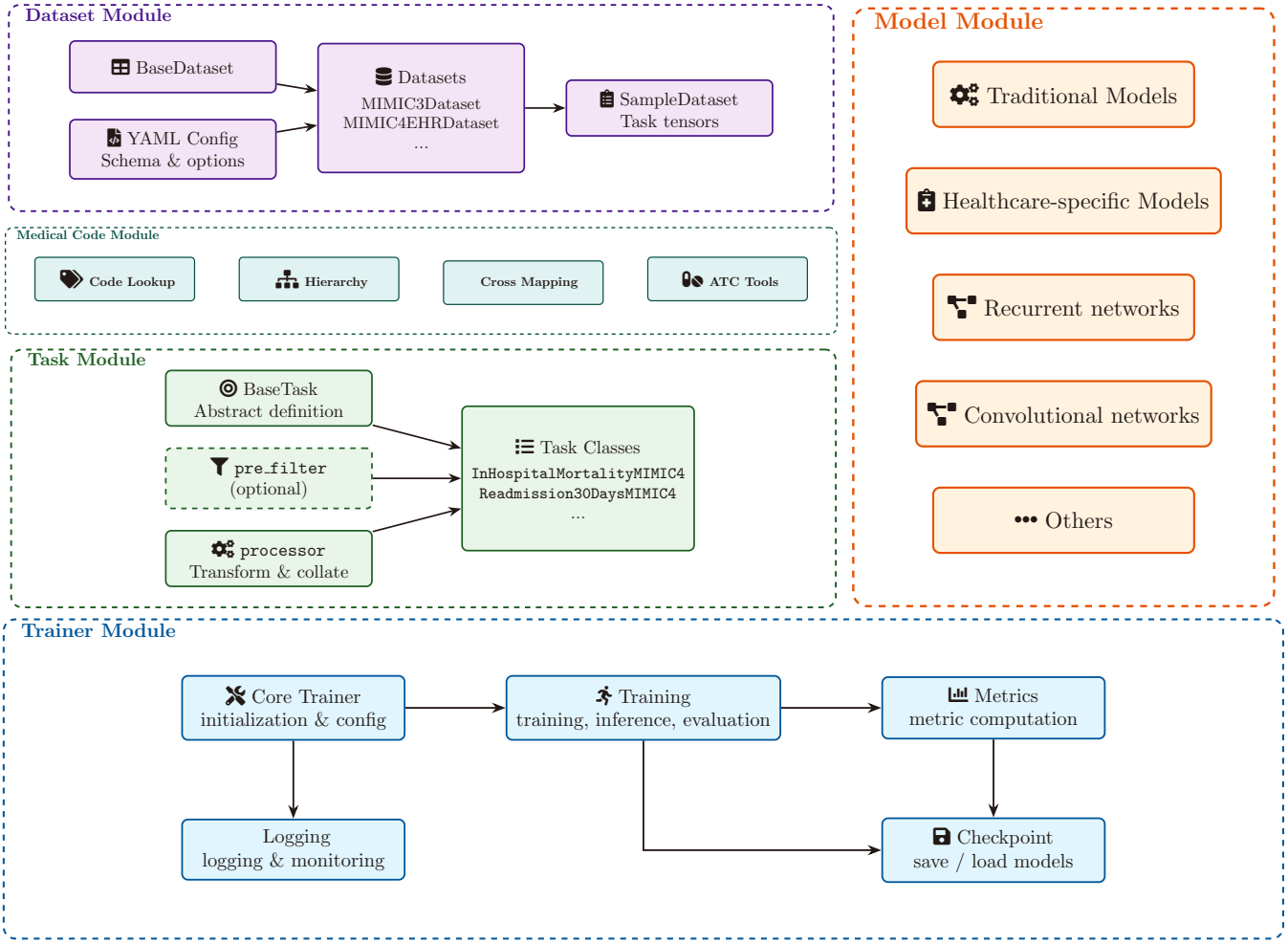


Fig. 1. Overview of the RHealth modular pipeline.

ing toolkit that brings the design philosophy and modularity of PyHealth into the R ecosystem for EHR predictive modeling. RHealth is engineered for convenience, low barrier to entry, integration, and modularity specifically for healthcare researchers and clinical data scientists. In a few lines of R, users can (i) harmonize multi-table EHR into patient-centric event streams with consistent coding, (ii) declare clinically meaningful tasks (e.g., in-hospital mortality, 30-day readmission) aligned with common benchmarks, (iii) train classical and neural models side-by-side (from Random Forest/XGBoost to LSTM/attention architectures), and (iv) evaluate models with reproducible splits, calibrated metrics, and transparent reports. By unifying dataset preparation, medical-code utilities, task specification, model interfaces, and trainer routines, the toolkit reduces "glue code," minimizes sources of leakage, and supports auditable end-to-end experiments—key prerequisites for reliability in clinical ML. In doing so, RHealth enables R-centric teams to participate fully in modern healthcare AI without ecosystem switching, while laying a stronger data-centric substrate on which responsible LLM applications can be built (e.g., using well-validated tabular predictions as inputs or guardrails to downstream language models). In short, RHealth operationalizes people-centric, trustworthy AI by meeting clinicians and clinical data scientists where they are—the R environment—while aligning with state-of-the-art methods across temporal modeling [3]–[8], clinical NLP [9]–[11], signal processing [12], and robust baselines [13], [14], thereby advancing accessible, reproducible, and reliable data science for better health. The code and documentation will be released after the double-blind review process.

## II. Toolkit Description

In a typical healthcare predictive modeling pipeline using EHR data, the journey from raw clinical records to a reliable prediction is complex and labor-intensive. Researchers must integrate heterogeneous tables (e.g., admissions, diagnoses, procedures), reconcile disparate medical coding systems, define outcome labels, and implement training routines – all steps that distract from the core research questions. Such intricacy slows progress for experts and poses a steep barrier for newcomers. The RHealth toolkit addresses these challenges

by structuring the workflow into five modular components (Dataset, Medical Code, Task, Model, Trainer; see Fig. 1), mirroring the pipeline from data ingestion to model training. Each module can be executed with only a few lines of R code while remaining fully customizable for advanced use. We detail each module below, including its motivation, design, implementation, and an illustrative example, to demonstrate how RHealth enables efficient and clinically relevant modeling (e.g., mortality or readmission prediction) with minimal overhead.

### A. Dataset Module

In routine EHR predictive modeling, raw clinical records are dispersed across multiple relational tables (e.g., patients, admissions, diagnoses, procedures, prescriptions) and inconsistent timestamp conventions. The *Dataset* module harmonizes these sources into a single, canonical event representation and materializes task-ready tensors, while providing practical accelerators such as CSV→Parquet caching (DuckDB) and a *dev mode* for fast iteration. These features reduce data-wrangling overhead and enable reproducible, large-scale pipelines within R.

In RHealth, datasets are instantiated through constructor classes such as `MIMIC4EHRDataset`, with configuration handled by a concise YAML schema that declares tables, identifiers, timestamps, and attributes. The resulting object exposes helper methods for data ingestion summaries (e.g., `$stats()`) and supports lazy, per-patient access for subsequent task construction.

The snippet below loads a MIMIC-IV demo cohort in *dev mode* and prints basic ingestion statistics.

```
# Load a demo EHR dataset (MIMIC-IV) with selected
tables
data_dir <- "/Users/yourname/datasets/mimiciv/"

ds <- MIMIC4EHRDataset$new(
  root = data_dir,
  tables = c("patients", "admissions",
             "diagnoses_icd", "procedures_icd",
             "prescriptions"),
  dataset_name = "mimic4_ehr",
  dev = TRUE   # use a smaller cohort for faster
  iteration
)

# Inspect ingestion summary (patients / events)
ds$stats()
#> Dataset : mimic4_ehr
#> Dev mode : TRUE
#> Patients : 1 000
#> Events   : 2 187 540
```

Patient-level access is enabled for debugging or exploratory analysis, allowing direct inspection of patient timelines with optional filtering by event type.

```
patient <- ds$get_patient("123456")
admissions <- patient$get_events(event_type =
"admissions")
length(admissions)
```

See Task/Debug notes for `get_patient()` and `get_events()`.

### B. Medical Code Module

Clinical EHR data contain rich coded information (diagnoses, medications, procedures) recorded in heterogeneous vocabularies such as ICD-9-CM, ICD-10-CM, ATC, and NDC. Researchers frequently need to interpret codes, traverse code hierarchies, or map between systems (e.g., consolidating multi-institution datasets or grouping fine-grained diagnoses into CCS categories). The *Medical Code* module provides a unified interface for medical code lookup and translation, abstracting multi-vocabulary complexity behind a consistent set of operations. Its design goal is to enable semantically consistent features and outcomes with minimal user effort, thereby improving clinical interpretability and downstream model reliability.

The module ships with (or can automatically retrieve) reference dictionaries for major coding systems and caches them locally for fast, offline queries after first use. Core operations include: (i) `lookup_code` for standardized descriptions and metadata; (ii) hierarchical traversal via `get_ancestors`/`get_descendants` to navigate code ontologies; and (iii) cross-system translation using `map_code` (e.g., ICD-9-CM → ICD-10-CM or ICD → CCS). The API is extensible: users can register additional vocabularies or custom mappings while retaining the same calling conventions. In practice, this normalization layer can be invoked during data preprocessing or feature engineering so that models consume consistent, clinically meaningful representations.

The snippet below illustrates common use cases: looking up an ICD-9-CM diagnosis code and mapping it to ICD-10-CM; then retrieving the corresponding higher-level CCS grouping.

```
# Lookup an ICD-9-CM diagnosis code
lookup_code("428.0", system = "ICD9CM")
# -> returns standardized description/metadata
(e.g., Congestive Heart Failure)

# Map ICD-9-CM -> ICD-10-CM
map_code("428.0", from = "ICD9CM", to = "ICD10CM")
# -> e.g., ["I50.30", "I50.31", ...] with official
ICD-10-CM descriptions

# Retrieve the broader CCS category for the code
map_code("428.0", from = "ICD9CM", to = "CCS")
# -> e.g., "CCS 108: Heart Failure"

# Explore hierarchy: list child (more specific)
codes under ICD-9 428
get_descendants("428", system = "ICD9CM")
# -> e.g., ["428.1", "428.20", ...]
```

By consolidating lookup, hierarchy navigation, and cross-system mapping into a single interface, the *Medical Code* module simplifies coded-data handling and promotes semantically coherent features for downstream modeling tasks.

### C. Task Module

Healthcare predictive modeling hinges on a precise definition of *what* to predict and *when* to predict it from a patient's longitudinal EHR. The *Task* module formalizes this process by decoupling problem specification from data ingestion and modeling. A task declares the prediction target (e.g., in-hospital mortality, 30-day readmission), the admissible input

window (e.g., first 48 h of encounters), and the output schema (binary/multi-class/continuous). Given a patient's event stream produced by the *Dataset* module, a task deterministically yields one or more model-ready training instances, ensuring consistent cohort construction, censoring rules, and label assignment across experiments.

Each task is implemented as an R6 class inheriting from a common `BaseTask`. The task exposes: (i) an `input_schema` describing feature layout (e.g., sequences of codes, aggregated labs), (ii) an `output_schema` describing the target type and shape, and (iii) a `call(patient)` method that inspects a single patient's chronologically ordered events and returns zero, one, or multiple `(x, y)` pairs according to the task's inclusion criteria and prediction time. Built-in tasks encode best practices such as *label leakage prevention* (excluding post-index events), *multiple-encounter handling*, and *robust missingness defaults*. Once instantiated, a task can be applied over the entire cohort to materialize a *SampleDataset* compatible with {`torch`} and the *Model/Trainer* stack. This separation makes task swaps and ablations (e.g., different horizons or alternative outcomes) straightforward and reproducible.

The snippet below illustrates the RHealth convention for defining an in-hospital mortality task: the first 48,h of data are used as input, and model-ready samples are then materialized from the dataset `ds`.

```
# Define a prediction task: in-hospital mortality @
48h
task <- InHospitalMortalityMIMIC4$new(
  input_window = "48h",        # admissible context
  label_name   = "mortality"   # binary target
)

# Apply task to dataset -> SampleDataset
(torch-compatible)
samples <- ds$set_task(task)

# Inspect shapes and label distribution (for sanity
checks)
print(samples$schema$input)     # feature layout
print(samples$schema$output)    # target spec
```

By centralizing labeling logic and schemas in a dedicated module, RHealth enables consistent task definitions across studies while reducing boilerplate and opportunities for subtle dataset shift or leakage.

### D. Model Moudle

Given prepared data and a defined prediction task, selecting and implementing a robust sequence model for longitudinal EHR is non-trivial in R. The *Model* module therefore provides *ready-to-use neural architectures* that are plug-and-play with RHealth datasets and tasks. All models share a common base class to ensure interface consistency and to minimize boilerplate.

Design goals are: (i) lower entry barrier with sensible defaults; (ii) automatic compatibility with the task's input/output schema; and (iii) easy extensibility for custom architectures.

All models inherit from `BaseModel`, which inspects the `SampleDataset` metadata to automatically infer input dimensions and output cardinality, select an appropriate loss function (e.g., binary cross-entropy for mortality, categorical cross-entropy for multi-class outcomes), and configure device placement (CPU/GPU). Reference implementations include an `RNN` for event sequences that embeds heterogeneous feature streams and aggregates temporal information before a task-specific output head. Models are implemented with {`torch`} for efficient training while exposing an R-native API. Users can subclass `BaseModel` to register custom layers/forward passes; the base class handles the "plumbing" (tensor shapes, loss, probability conversion), preserving compatibility with the `Trainer`.

The snippet below instantiates a default `RNN` for the task-specific dataset; only high-level hyperparameters are required, while dimensions and losses are inferred automatically.

```
# Initialize a default RNN model for the given
task/dataset
model <- RNN(
  dataset       = samples,
  embedding_dim = 128,
  hidden_dim    = 128
)
```

This modular interface allows researchers to switch architectures (e.g., from `RNN` to a custom model) without changing upstream data/task code or downstream training procedures, thereby maintaining a clean separation of concerns across the RHealth pipeline.

### E. Trainer Module

Training deep learning models on EHR data entails orchestrating optimization, monitoring performance, saving checkpoints, and enforcing robustness—tasks that are tedious and error-prone if reimplemented for every study. RHealth provides a high-level, configurable *Trainer* that encapsulates these concerns and exposes sensible defaults (logging, validation, early stopping), while allowing experts to fine-tune behavior. By handling boilerplate reliably, the Trainer lets researchers focus on modeling choices and clinical interpretation rather than engineering the training loop.

The Trainer is an R6 class that accepts any `BaseModel`-compatible model and a pair of data loaders for training/validation. Upon initialization it configures an experiment directory and logging utilities (hyper-parameters, losses, metrics). The main routine `train()` performs multi-epoch optimization with periodic validation, supports early stopping and "best model" tracking by a monitored metric (e.g., validation ROC AUC), and checkpoints weights accordingly.

Good practices such as gradient clipping and separated weight decay (excluding biases/normalization parameters) are built in. The Trainer integrates seamlessly with {*torch*} backends for CPU/GPU acceleration and accepts user-defined metrics (e.g., AUROC, AUPRC) to ensure clinically meaningful evaluation.

The following code snippet shows a typical workflow: splitting patient-level samples, instantiating an RNN, and launching training while monitoring validation ROC AUC.

```
# Split by patient and build data loaders
splits   <- split_by_patient(samples, ratios =
c(0.8, 0.2), stratify = TRUE)
train_dl <- get_dataloader(splits[[1]], batch_size =
32, shuffle = TRUE)
val_dl   <- get_dataloader(splits[[2]],  batch_size
= 32)

# Initialize model and Trainer
model   <- RNN(dataset = samples, embedding_dim =
128, hidden_dim = 128)
trainer <- Trainer$new(
  model        = model,
  metrics      = c("auroc", "auprc"),
  output_path  = "experiments/",
  exp_name     = "mortality_rnn"
)

# Run training with validation monitoring and
checkpointing
trainer$train(
  train_dataloader = train_dl,
  val_dataloader   = val_dl,
  epochs           = 10,
  optimizer_params = list(lr = 1e-3),
  monitor          = "roc_auc"   # early stopping /
  best model by val ROC AUC
)
```

During training, the Trainer logs losses/metrics per epoch, maintains a checkpoint of the best-performing model, and restores it upon completion. This modular abstraction yields reliable, reproducible training runs with minimal user code while preserving expert control when needed.

## III. DISCUSSION

RHealth is designed explicitly for healthcare researchers and clinical data scientists who work primarily in R and need to build reliable EHR prediction pipelines without switching ecosystems. By unifying dataset harmonization, medical-code utilities, task specification, model interfaces, and training routines, the toolkit reduces "glue code," lowers the barrier to entry, and makes auditable, end-to-end experimentation feasible with minimal scripting. In practice, this translates into faster hypothesis iteration (e.g., mortality vs. readmission), easier ablations (e.g., input horizons, label variants), and reproducible evaluations with calibrated metrics. The modular architecture also enables side-by-side comparison of classical learners (Random Forests, gradient boosting) and deep sequential models (RNN/attention), which many clinical teams require to balance performance, interpretability, and operational constraints. In the context of people-centric and LLM-enabled healthcare, RHealth serves as a dependable upstream substrate: well-curated, consistently coded, and rigorously evaluated tabular predictions can inform, constrain, or audit downstream language-model agents.

There are, however, important limitations. First, breadth: while the current design covers common EHR table families and coding systems, broader connectors (e.g., OMOP/CDM variants), richer multi-modal support (signals, imaging, and notes), and task templates beyond classification (e.g., time-to-event, treatment-effect modeling) warrant expansion. Second, trustworthiness: systematic uncertainty estimation, robustness testing under distribution shift, and bias/fairness audits must be made first-class citizens with standardized reports. Third, generalization: stronger cross-site validation tooling (federated or privacy-preserving options) and plug-and-play data governance hooks (de-identification checks, lineage, and access logging) are needed for translation into practice. Finally, to sustain impact, an open community process (benchmarks, model cards, error taxonomies) is essential to ensure results remain comparable across cohorts and institutions. We view RHealth as a practical step toward these goals: a compact, R-native foundation that foregrounds accessibility, modularity, and integration while leaving clear growth paths for reliability-critical extensions and LLM-era workflows.

## IV. CONCLUSION

We presented RHealth, an open-source, R-native deep learning toolkit that operationalizes reliable EHR modeling for healthcare researchers and clinical data scientists. By providing a coherent pipeline—from multi-table data harmonization and code mapping to task definition, model training, and calibrated evaluation—RHealth reduces engineering overhead and improves reproducibility and auditability. This accessibility matters for people-centric AI: it empowers domain experts to prototype, compare, and validate models in their familiar analytical environment, and to feed trustworthy, well-evaluated signals into emerging LLM applications. Looking forward, we will extend RHealth toward multi-modal data, uncertainty and fairness reporting, cross-site generalization tooling, and privacy-preserving training—aligning the toolkit with the workshop's vision of advancing data for better health. In doing so, RHealth helps bridge methodological advances and clinical utility, enabling responsible, data-centric AI that is easier to adopt, scrutinize, and sustain in real-world healthcare.

## V. ACKNOWLEDGEMENT

This work was supported by the Infrastructure Steering Committee (ISC) grant from the R Consortium. Junyi Gao acknowledged the receipt of studentship awards from the Health Data Research UK-The Alan Turing Institute Wellcome PhD Programme in Health Data Science (grant 218529/Z/19/Z) and Baidu Scholarship.

## REFERENCES

[1] C. Xiao, E. Choi, and J. Sun, "Opportunities and challenges in developing deep learning models using electronic health records data: a systematic review," *Journal of the American Medical Informatics Association*, vol. 25, no. 10, pp. 1419–1428, 06 2018. [Online]. Available: https://doi.org/10.1093/jamia/ocy068

[2] H. Harutyunyan, H. Khachatrian, D. C. Kale, G. Ver Steeg, and A. Galstyan, "Multitask learning and benchmarking with clinical time series data," *Scientific Data*, vol. 6, no. 1, p. 96, 2019. [Online]. Available: https://doi.org/10.1038/s41597-019-0103-9

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.

[5] I. M. Baytas, C. Xiao, X. Zhang, F. Wang, A. K. Jain, and J. Zhou, "Patient subtyping via time-aware lstm networks," in *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, 2017, pp. 65–74.

[6] E. Choi, M. T. Bahadori, J. Sun, J. Kulas, A. Schuetz, and W. Stewart, "Retain: An interpretable predictive model for healthcare using reverse time attention mechanism," *Advances in neural information processing systems*, vol. 29, 2016.

[7] J. Gao, C. Xiao, Y. Wang, W. Tang, L. M. Glass, and J. Sun, "Stagenet: Stage-aware neural networks for health risk prediction," in *Proceedings of the web conference 2020*, 2020, pp. 530–540.

[8] Y. Xu, S. Biswal, S. R. Deshpande, K. O. Maher, and J. Sun, "Raim: Recurrent attentive and intensive model of multimodal patient monitoring data," in *Proceedings of the 24th ACM SIGKDD international conference on Knowledge Discovery & Data Mining*, 2018, pp. 2565–2573.

[9] J. Mullenbach, S. Wiegreffe, J. Duke, J. Sun, and J. Eisenstein, "Explainable prediction of medical codes from clinical text," *arXiv preprint arXiv:1802.05695*, 2018.

[10] T. Vu, D. Q. Nguyen, and A. Nguyen, "A label attention model for icd coding from clinical text," *arXiv preprint arXiv:2007.06351*, 2020.

[11] F. Li and H. Yu, "Icd coding from clinical text using multi-filter residual convolutional neural network," in *proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 8180–8187.

[12] Z. Li, D. Zhou, L. Wan, J. Li, and W. Mou, "Heartbeat classification using deep residual convolutional neural network from 2-lead electrocardiogram," *Journal of electrocardiology*, vol. 58, pp. 105–112, 2020.

[13] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[14] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.

[15] Y. Zhao, Z. Qiao, C. Xiao, L. Glass, and J. Sun, "Pyhealth: A python library for health predictive models," *arXiv preprint arXiv:2101.04209*, 2021.